

Q1. Create a class `Calculator` that demonstrates method overloading by performing operations on different data types (int, float, double) and varying number of parameters.

```
Lab_10 > Q1.java > ...
1  package Lab_10;
2  class Calculator {
3
4      int add(int a, int b) {
5          return a + b;
6      }
7
8      float add(float a, float b) {
9          return a + b;
10     }
11
12     double add(double a, double b, double c) {
13         return a + b + c;
14     }
15 }
16
17 public class Q1 {
18     Run | Debug
19     public static void main(String[] args) {
20
21         Calculator c = new Calculator();
22
23         System.out.println("Int Addition: " + c.add(a: 10, b: 20));
24
25         System.out.println("Float Addition: " + c.add(a: 10.5f, b: 20.5f));
26
27         System.out.println("Double Addition: " + c.add(a: 1.1, b: 2.2, c: 3.3));
28     }
29 }
```

OUTPUT:—

```
gauravmathur@Annus-Mac LabJava % javac Lab_10/Q1.java
gauravmathur@Annus-Mac LabJava % java Lab_10.Q1
Int Addition: 30
Float Addition: 31.0
Double Addition: 6.6
gauravmathur@Annus-Mac LabJava %
```

Q2. Develop a program where a superclass defines a generic method (e.g., `displayDetails()`), and subclasses provide specialized implementations.

```
Lab_10 > Q2.java > Q2
1  package Lab_10;
2  class Person {
3      void displayDetails() {
4          System.out.println(x: "Displaying Person Details");
5      }
6  }
7  class Student extends Person {
8      void displayDetails() {
9          System.out.println(x: "Displaying Student Details");
10     }
11 }
12 class Teacher extends Person {
13     void displayDetails() {
14         System.out.println(x: "Displaying Teacher Details");
15     }
16 }
17 public class Q2 {
18     public static void main(String[] args) {
19         Student s = new Student();
20         Teacher t = new Teacher();
21
22         s.displayDetails();
23         t.displayDetails();
24     }
25 }
26 }
```

```
Double Addition: 0.0
● gauravmathur@Annus-Mac LabJava % javac Lab_10/Q2.java
● gauravmathur@Annus-Mac LabJava % java Lab_10.Q2
Displaying Student Details
Displaying Teacher Details
gauravmathur@Annus-Mac LabJava %
```

Q3. Write a program to demonstrate that **method overriding requires same method signature** and analyze cases where it fails (modifying parameters).

```
Lab_10 > Q3.java > Q3
1  package Lab_10;
2  class Parent {
3
4      void show(int a) {
5          System.out.println(x: "Parent Method");
6      }
7  }
8
9  class Child extends Parent {
10
11      // This is NOT overriding, it is overloading
12      void show() {
13          System.out.println(x: "Child Method");
14      }
15  }
16
17  public class Q3 {
18
19      Run | Debug
20      public static void main(String[] args) {
21
22          Child c = new Child();
23
24          c.show();
25          c.show(a: 10);
26      }
27  }
```

OUTPUT:—

```
-----
Displaying Teacher Details
● gauravmathur@Annus-Mac LabJava % javac Lab_10/Q3.java
● gauravmathur@Annus-Mac LabJava % java Lab_10.Q3
Child Method
Parent Method
gauravmathur@Annus-Mac LabJava %
Ready
```

Q4. Write a program to demonstrate **method overriding with covariant return types** (child class returning a more specific type than parent).

```
Lab_10 > Q4.java > Q4
1  package Lab_10;
2  class Animal {
3
4      Animal getObject() {
5          System.out.println(x: "Animal Object");
6          return this;
7      }
8  }
9
10 class Dog extends Animal {
11
12     Dog getObject() {
13         System.out.println(x: "Dog Object");
14         return this;
15     }
16 }
17
18 public class Q4 {
19
20     Run | Debug
21     public static void main(String[] args) {
22
23         Dog d = new Dog();
24         d.getObject();
25     }
26 }
```

OUTPUT:—

```
parent method
● gauravmathur@Annus-Mac LabJava % javac Lab_10/Q4.java
● gauravmathur@Annus-Mac LabJava % java Lab_10.Q4
Dog Object
gauravmathur@Annus-Mac LabJava %
```

Q5. Develop a program to show that private methods cannot be overridden and analyze the behavior.

```
Lab_10 > Q5.java > Q5
1  package Lab_10;
2  class Parent {
3
4      private void display() {
5          System.out.println(x: "Private Method of Parent");
6      }
7
8      void show() {
9          display();
10     }
11 }
12
13 class Child extends Parent {
14
15     void display() {
16         System.out.println(x: "Method of Child");
17     }
18 }
19
20 public class Q5 {
21
22     Run | Debug
23     public static void main(String[] args) {
24
25         Child c = new Child();
26
27         c.display();
28         c.show();
29     }
30 }
```

OUTPUT:—

```
log object
● gauravmathur@Annus-Mac LabJava % javac Lab_10/Q5.java
● gauravmathur@Annus-Mac LabJava % java Lab_10.Q5
Method of Child
Private Method of Parent
❖ gauravmathur@Annus-Mac LabJava %
```

Q6. Create a program to demonstrate overriding rules related to:

- access modifiers (increasing visibility)
- changing the return types
- parameter consistency

```
Lab_10 > Q6.java > Q6
1  package Lab_10;
2  class Parent {
3
4      protected Number show(int a) {
5          System.out.println(x: "Parent Method");
6          return a;
7      }
8  }
9
10 class Child extends Parent {
11
12     // Increased visibility: protected -> public
13     // Covariant return type: Integer is subclass of Number
14     public Integer show(int a) {
15
16         System.out.println(x: "Child Method");
17
18         return a;
19     }
20 }
21
22 public class Q6 {
23
24     Run | Debug
25     public static void main(String[] args) {
26
27         Child c = new Child();
28
29         System.out.println(c.show(a: 100));
30     }
31 }
```

PROBLEMS 5 OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
gauravmathur@Annus-Mac LabJava % javac Lab_10/Q6.java
gauravmathur@Annus-Mac LabJava % java Lab_10.Q6
Child Method
100
gauravmathur@Annus-Mac LabJava %
```